Value-Based Model Free RL

Last Time

- Policy Optimization
- Policy Gradient
- Tricks for Policy Gradient

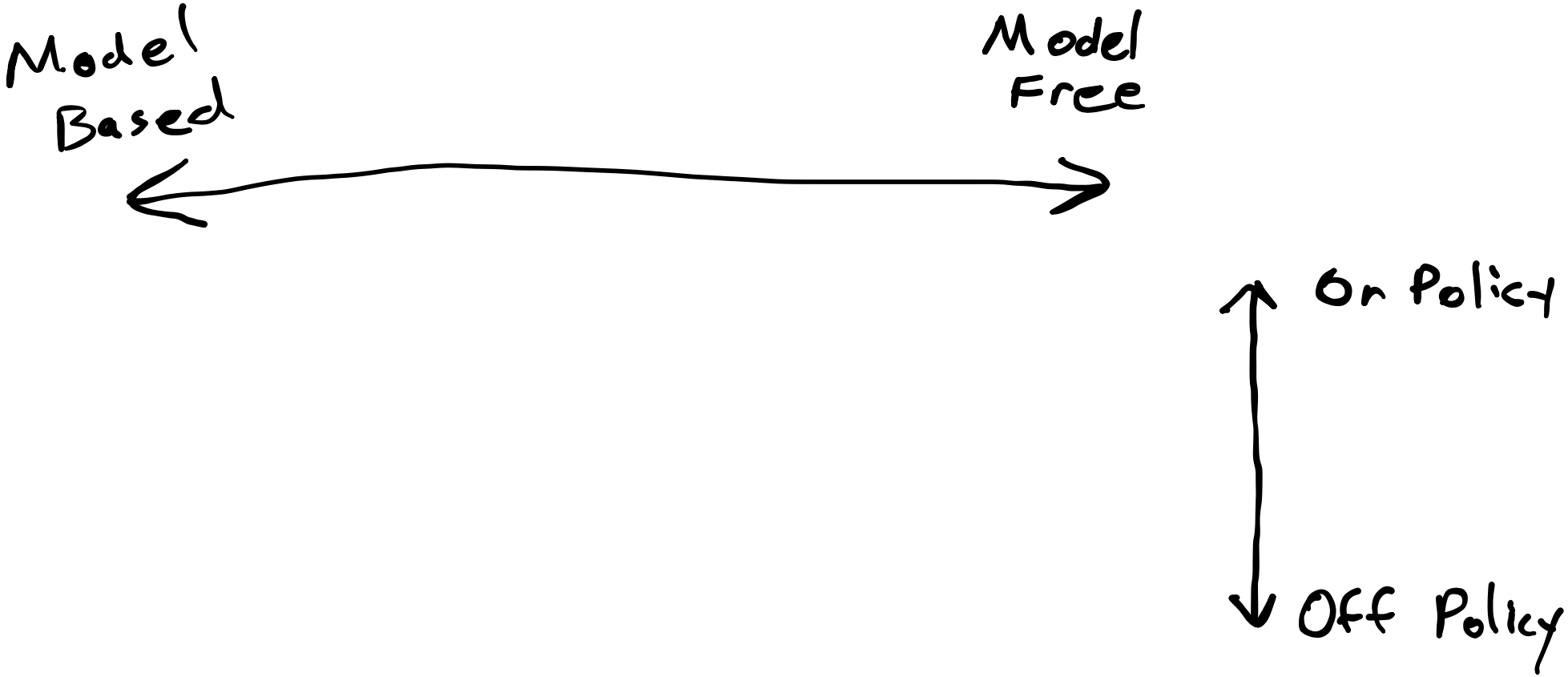
Map

Map

Model Based

Model Free

Map



Map

Model Based ⟷ Model Free

On Policy ↕ Off Policy

MLMBTRL
(learn T,R)

Map

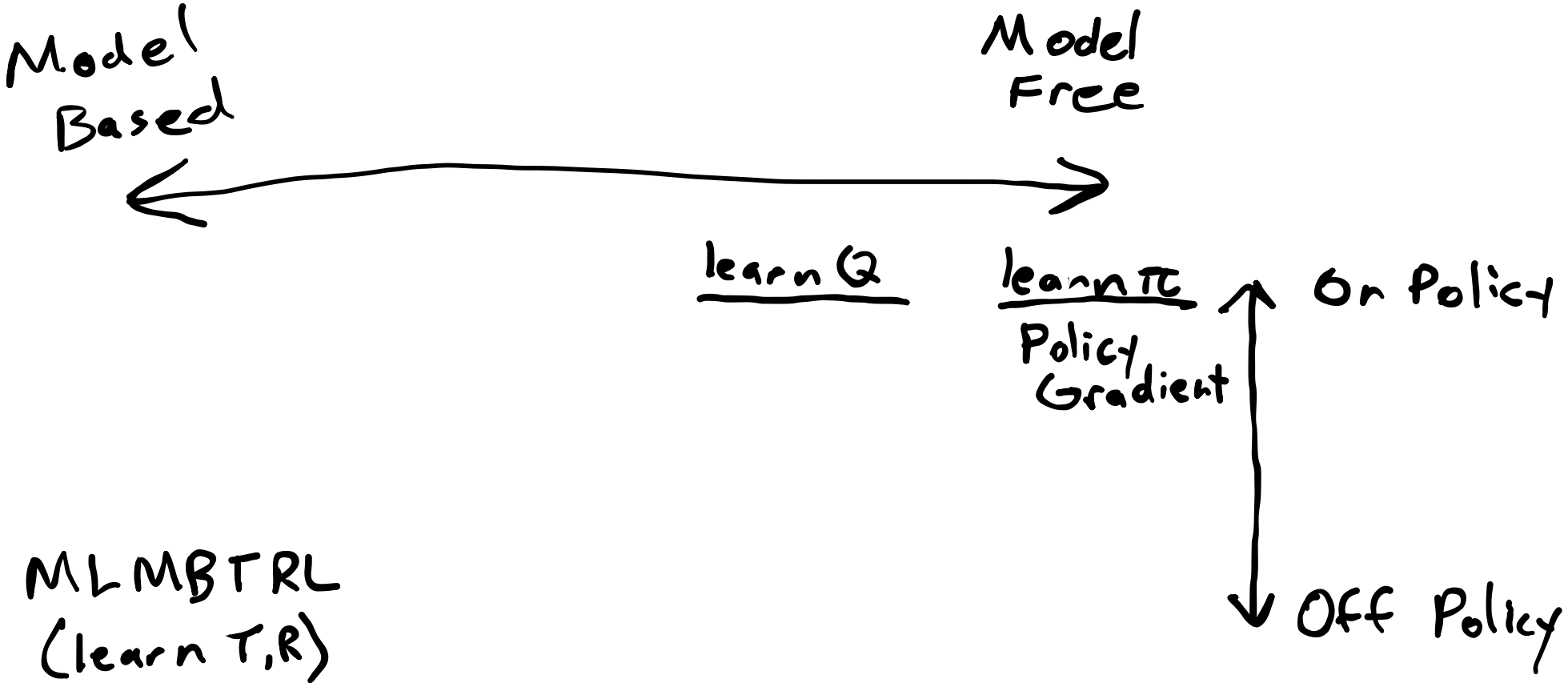
Model Based ⟷ Model Free

learn Q learn π

On Policy ↕ Off Policy

MLMBTRL
(learn T, R)

Map


Model Based
Model Free
learn Q
learn π
Policy Gradient
On Policy
Off Policy
MLMBTRL
(learn T,R)

Map

Model Based ⟷ Model Free

learn Q

SARSA

learn π

Policy Gradient

On Policy

Off Policy

MLMBTRL
(learn T, R)

Map

Model Based ⟷ Model Free

learn Q

SARSA

learn π

Policy Gradient

On Policy

Off Policy

MLMBTRL
(learn T, R)

Q-learning

Today

Today

- Basic On- and Off-Policy **value based** model free RL algorithms

Today

- Basic On- and Off-Policy **value based** model free RL algorithms
- Tricks for tabular value based RL algorithms

Today

- Basic On- and Off-Policy **value based** model free RL algorithms
- Tricks for tabular value based RL algorithms
- Understanding of On- vs Off-Policy

Why learn Q?

T, R

Big

π

Hard to interpret θ

Local optimum

On Policy → Poor sample complexity

More step

$U(s)$ $Q(s,a)$

$\pi(s) = \text{argmax}\ Q(s,a)$

$\pi(s) = \text{argmax}\left(R(s,a) + \gamma E[U(s')]\right)$

Incremental Mean Estimation

Incremental Mean Estimation

$$\hat{\boldsymbol{x}}_m = \frac{1}{m} \sum_{i=1}^{m} \boldsymbol{x}^{(i)}$$

Incremental Mean Estimation

$$
\begin{aligned}
\hat{\boldsymbol{x}}_m &= \frac{1}{m} \sum_{i=1}^{m} \boldsymbol{x}^{(i)} \\
&= \frac{1}{m} \left(\boldsymbol{x}^{(m)} + \sum_{i=1}^{m-1} \boldsymbol{x}^{(i)} \right)
\end{aligned}
$$

Incremental Mean Estimation

$$
\begin{aligned}
\hat{x}_m &= \frac{1}{m} \sum_{i=1}^{m} x^{(i)} \\
&= \frac{1}{m} \left(x^{(m)} + \sum_{i=1}^{m-1} x^{(i)} \right) \\
&= \frac{1}{m} \left(x^{(m)} + (m-1)\, \hat{x}_{m-1} \right)
\end{aligned}
$$

Incremental Mean Estimation

$$
\begin{aligned}
\hat{x}_m &= \frac{1}{m} \sum_{i=1}^{m} x^{(i)} \\
&= \frac{1}{m} \left(x^{(m)} + \sum_{i=1}^{m-1} x^{(i)} \right) \\
&= \frac{1}{m} \left(x^{(m)} + (m-1)\, \hat{x}_{m-1} \right) \\
&= \hat{x}_{m-1} + \frac{1}{m} \left(x^{(m)} - \hat{x}_{m-1} \right)
\end{aligned}
$$

Incremental Mean Estimation

$$
\begin{aligned}
\hat{x}_m &= \frac{1}{m}\sum_{i=1}^{m} x^{(i)} \\
&= \frac{1}{m}\left(x^{(m)} + \sum_{i=1}^{m-1} x^{(i)}\right) \\
&= \frac{1}{m}\left(x^{(m)} + (m-1)\,\hat{x}_{m-1}\right) \\
&= \hat{x}_{m-1} + \frac{1}{m}\left(x^{(m)} - \hat{x}_{m-1}\right)
\end{aligned}
$$

```julia
function simulate!(π::MonteCarloTreeSearch, s, d=π.d)
    if d ≤ 0
        return π.U(s)
    end
    𝒫, N, Q, c = π.𝒫, π.N, π.Q, π.c
    𝒜, TR, γ = 𝒫.𝒜, 𝒫.TR, 𝒫.γ
    if !haskey(N, (s, first(𝒜)))
        for a in 𝒜
            N[(s,a)] = 0
            Q[(s,a)] = 0.0
        end
        return π.U(s)
    end
    a = explore(π, s)
    s′, r = TR(s,a)
    q = r + γ*simulate!(π, s′, d-1)
    N[(s,a)] += 1
    Q[(s,a)] += (q-Q[(s,a)])/N[(s,a)]
    return q
end
```

Incremental Mean Estimation

$$
\begin{aligned}
\hat{x}_m &= \frac{1}{m}\sum_{i=1}^{m} x^{(i)} \\
&= \frac{1}{m}\left(x^{(m)} + \sum_{i=1}^{m-1} x^{(i)}\right) \\
&= \frac{1}{m}\left(x^{(m)} + (m-1)\,\hat{x}_{m-1}\right) \\
&= \hat{x}_{m-1} + \frac{1}{m}\left(x^{(m)} - \hat{x}_{m-1}\right)
\end{aligned}
$$

```julia
function simulate!(π::MonteCarloTreeSearch, s, d=π.d)
    if d ≤ 0
        return π.U(s)
    end
    𝒫, N, Q, c = π.𝒫, π.N, π.Q, π.c
    𝒜, TR, γ = 𝒫.𝒜, 𝒫.TR, 𝒫.γ
    if !haskey(N, (s, first(𝒜)))
        for a in 𝒜
            N[(s,a)] = 0
            Q[(s,a)] = 0.0
        end
        return π.U(s)
    end
    a = explore(π, s)
    s′, r = TR(s,a)
    q = r + γ*simulate!(π, s′, d-1)
    Q[(s,a)] += (q-Q[(s,a)])/N[(s,a)]
end
```

Incremental Mean Estimation

$$
\begin{aligned}
\hat{x}_m &= \frac{1}{m}\sum_{i=1}^{m} x^{(i)} \\
&= \frac{1}{m}\left(x^{(m)} + \sum_{i=1}^{m-1} x^{(i)}\right) \\
&= \frac{1}{m}\left(x^{(m)} + (m-1)\,\hat{x}_{m-1}\right) \\
&= \hat{x}_{m-1} + \frac{1}{m}\left(x^{(m)} - \hat{x}_{m-1}\right)
\end{aligned}
$$

```julia
function simulate!(π::MonteCarloTreeSearch, s, d=π.d)
    if d ≤ 0
        return π.U(s)
    end
    𝒫, N, Q, c = π.𝒫, π.N, π.Q, π.c
    𝒜, TR, γ = 𝒫.𝒜, 𝒫.TR, 𝒫.γ
    if !haskey(N, (s, first(𝒜)))
        for a in 𝒜
            N[(s,a)] = 0
            Q[(s,a)] = 0.0
        end
        return π.U(s)
    end
    a = explore(π, s)
    s′, r = TR(s,a)
    q = r + γ*simulate!(π, s′, d-1)
    Q[(s,a)] += (q-Q[(s,a)])/N[(s,a)]
end
```

loop

$$\hat{x} \leftarrow \hat{x} + \alpha\left(x - \hat{x}\right)$$

Incremental Mean Estimation

$$
\begin{aligned}
\hat{x}_m &= \frac{1}{m}\sum_{i=1}^{m} x^{(i)} \\
&= \frac{1}{m}\left(x^{(m)} + \sum_{i=1}^{m-1} x^{(i)}\right) \\
&= \frac{1}{m}\left(x^{(m)} + (m-1)\,\hat{x}_{m-1}\right) \\
&= \hat{x}_{m-1} + \frac{1}{m}\left(x^{(m)} - \hat{x}_{m-1}\right)
\end{aligned}
$$

```julia
function simulate!(π::MonteCarloTreeSearch, s, d=π.d)
    if d ≤ 0
        return π.U(s)
    end
    𝒫, N, Q, c = π.𝒫, π.N, π.Q, π.c
    𝒜, TR, γ = 𝒫.𝒜, 𝒫.TR, 𝒫.γ
    if !haskey(N, (s, first(𝒜)))
        for a in 𝒜
            N[(s,a)] = 0
            Q[(s,a)] = 0.0
        end
        return π.U(s)
    end
    a = explore(π, s)
    s′, r = TR(s,a)
    q = r + γ*simulate!(π, s′, d-1)
    Q[(s,a)] += (q-Q[(s,a)])/N[(s,a)]
end
```

loop

Learning

$$\hat{x} \leftarrow \hat{x} + \alpha\,(x - \hat{x})$$

"Temporal Difference (TD) Error"

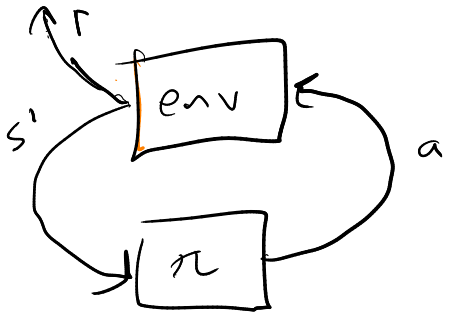
Q Learning

Table $Q(s,a)$

Want $Q(s,a) \leftarrow Q(s,a) + \alpha\left(q(s,a,r,s') - Q(s,a)\right)$

$$Q(s,a) = R(s,a) + \gamma \underset{s' \sim T(s'|s,a)}{E}\left[U(s')\right]$$

$$= \underbrace{R(s,a)}_{\text{don't know}} + \gamma \underbrace{\underset{s' \sim T(s'|s,a)}{E}}_{?}\left[\max_{a'} Q(s',a')\right]$$



$(s,a,r,s') \leftarrow$ experience tuple

$$q(s,a,r,s') = r + \gamma \max_{a'} Q(s',a')$$

$$Q(s,a) \leftarrow Q(s,a) + \alpha\left(r + \gamma \max_{a'} Q(s',a') - Q(s,a)\right)$$

Q learning and SARSA

Q learning and SARSA

<u>Q-Learning</u>

$Q(s, a) \leftarrow 0$

$s \leftarrow s_0$

loop

$a \leftarrow \text{argmax}\, Q(s, a) \underbrace{\text{w.p. } 1 - \epsilon, \quad \text{rand}(A)}_{\varepsilon\text{-greedy}} \text{ o.w.}$

$r \leftarrow \text{act!}(\text{env}, a)$

$s' \leftarrow \text{observe}(\text{env})$

$Q(s, a) \leftarrow Q(s, a) + \alpha\ (r + \gamma \max_{a'} Q(s', a') - Q(s, a))$

$s \leftarrow s'$

Q learning and SARSA

Q-Learning

(s, a, r, s')

$Q(s,a) \leftarrow 0$

$s \leftarrow s_0$

loop

$a \leftarrow \text{argmax}\, Q(s,a)$ w.p. $1-\epsilon$, $\quad \text{rand}(A)$ o.w.

with probability otherwise

$r \leftarrow \text{act!}(\text{env}, a)$

$s' \leftarrow \text{observe}(\text{env})$

$Q(s,a) \leftarrow Q(s,a) + \alpha\,(\underbrace{r + \gamma \max_{a'} Q(s',a') - Q(s,a)}_{\text{TD}})$

$s \leftarrow s'$

SARSA

(s, a, r, s', a')

comes from exploration policy

$a \leftarrow \text{rand}(A)$

X

$a' \leftarrow$ exploration policy (e.g. eps. greedy

$Q(s,a) \leftarrow Q(s,a) + \alpha(r + \gamma Q(s',a') - Q(s,a))$

$a \leftarrow a'$

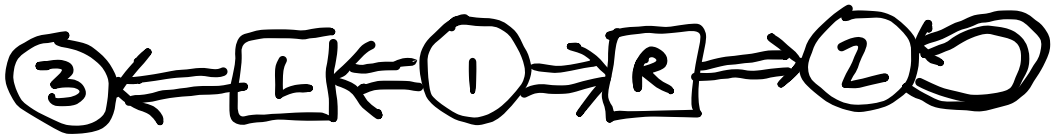
Learning $Q^{\pi_{\text{exploration}}}$

Illustrative Problem

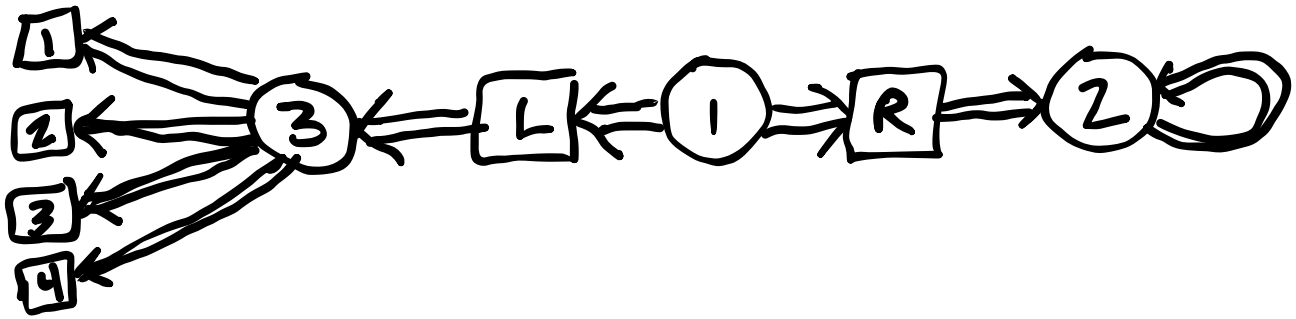
Illustrative Problem

Illustrative Problem

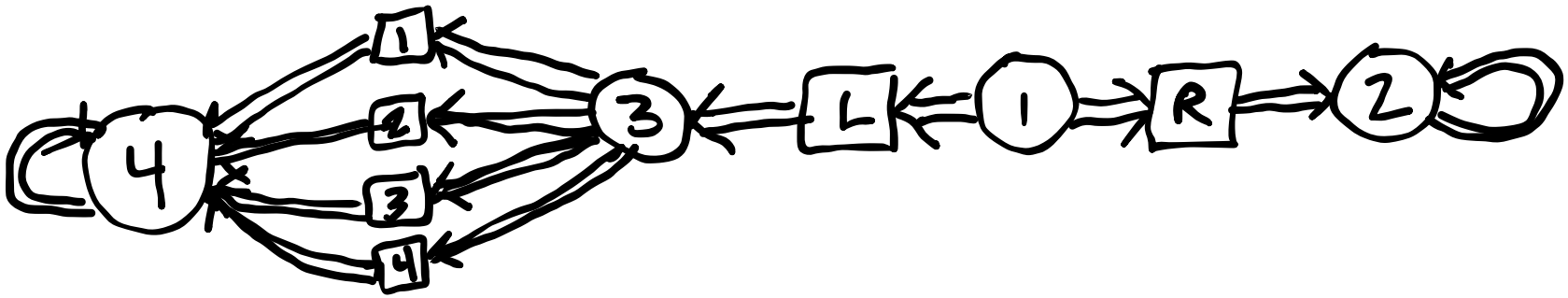
Illustrative Problem



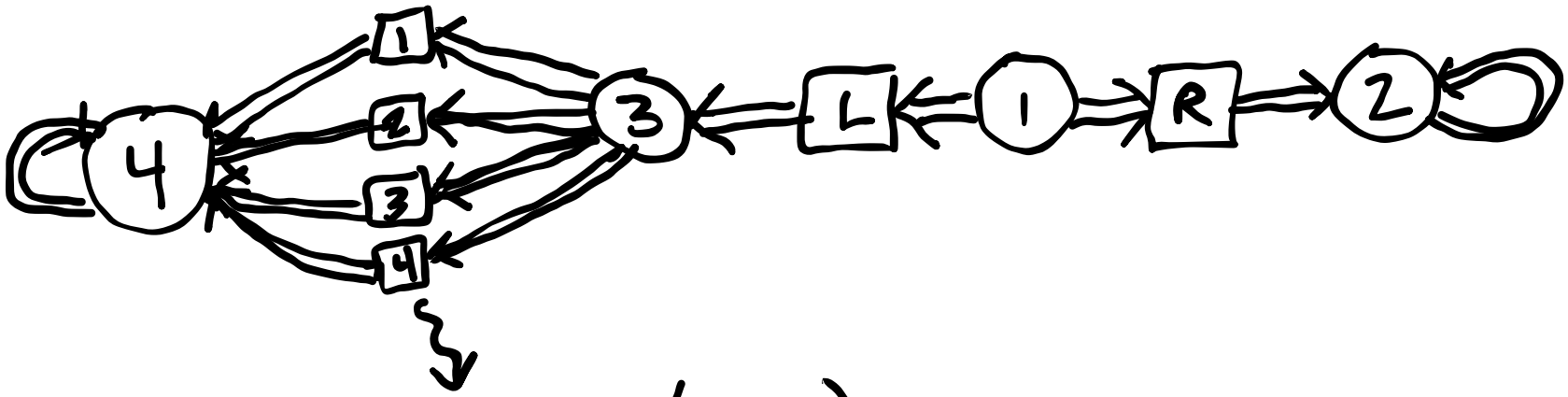
Illustrative Problem



Illustrative Problem

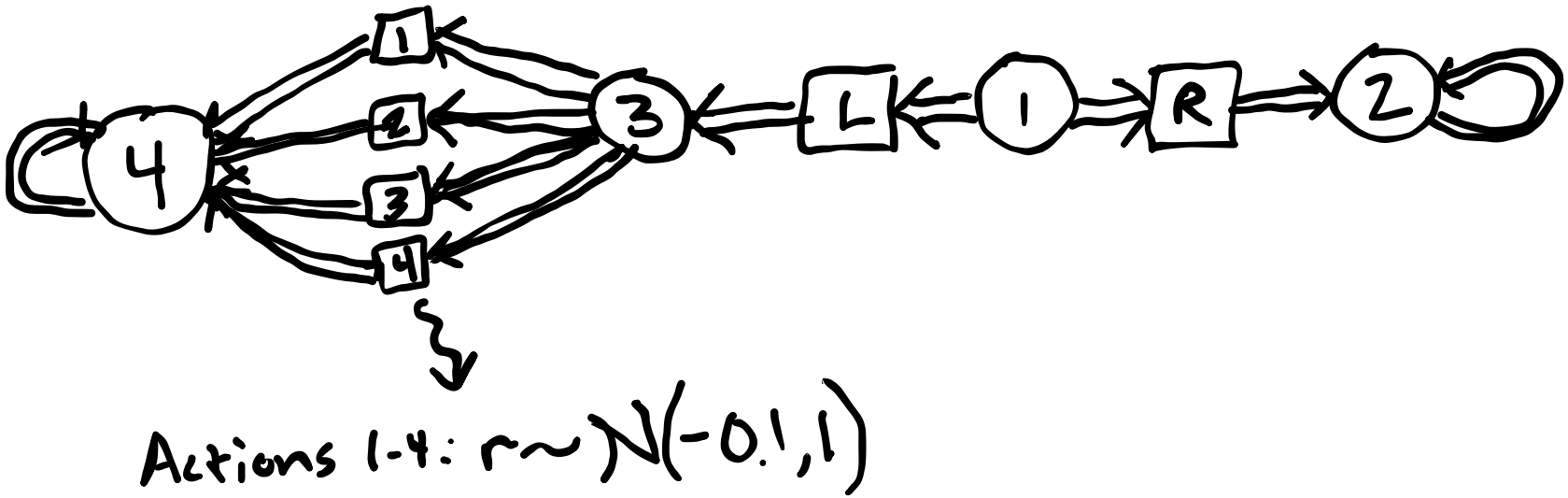


Illustrative Problem



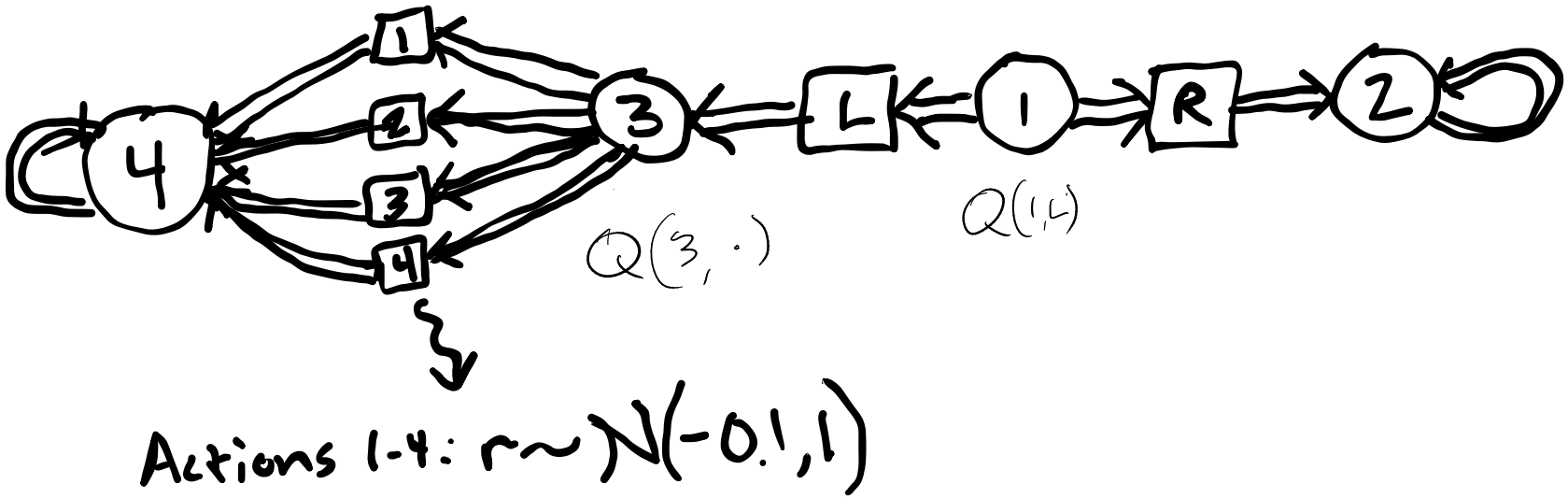
Actions 1-4: $r \sim N(-0.1, 1)$

Illustrative Problem



1. After a few episodes, what is Q(3, a) for a in 1-4?

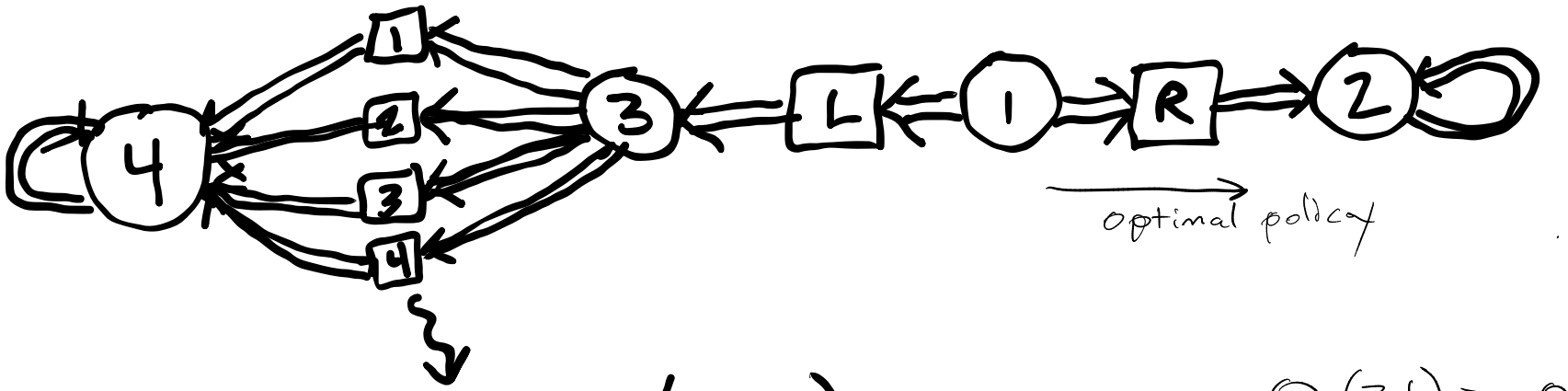
Illustrative Problem



1. After a few episodes, what is Q(3, a) for a in 1-4?
2. After a few episodes, what is Q(1, L)?

Illustrative Problem

$Q(1,R) = 0$



Actions 1-4: $r \sim N(-0.1, 1)$

$Q(3,1) = -0.1 \pm 0.2$

$Q(3,2) = -0.1 \pm 0.2$

$Q(3,3) = \vdots$

$Q(3,4) = \vdots$

Q Learning algorithm

1. After a few episodes, what is Q(3, a) for a in 1-4?
2. After a few episodes, what is Q(1, L)?
3. Why is this a problem and what are some possible solutions?

$$Q(1,L) \leftarrow Q(1,L) + \alpha\left(r + \gamma \max_{a'} Q(3,a') - Q(1,L)\right)$$

> 0

Big Problem: Maximization Bias

Big Problem: Maximization Bias

Even if all $Q(s', a')$ unbiased, $\max_{a'} Q(s', a')$ is biased!

Big Problem: Maximization Bias

Even if all $Q(s', a')$ unbiased, $\max_{a'} Q(s', a')$ is biased!

Solution: Double Q Learning

Big Problem: Maximization Bias

Even if all $Q(s', a')$ unbiased, $\max_{a'} Q(s', a')$ is biased!

Solution: Double Q Learning Q_1, Q_2

Big Problem: Maximization Bias

Even if all $Q(s', a')$ unbiased, $\max_{a'} Q(s', a')$ is biased!

Solution: Double Q Learning $\quad Q_1, Q_2$

$$Q_1(s,a) \leftarrow Q_1(s,a) + \alpha \left(r + \gamma Q_2 \left(s', \underset{a'}{\operatorname{argmax}} Q_1(s', a') \right) - Q_1(s,a) \right)$$

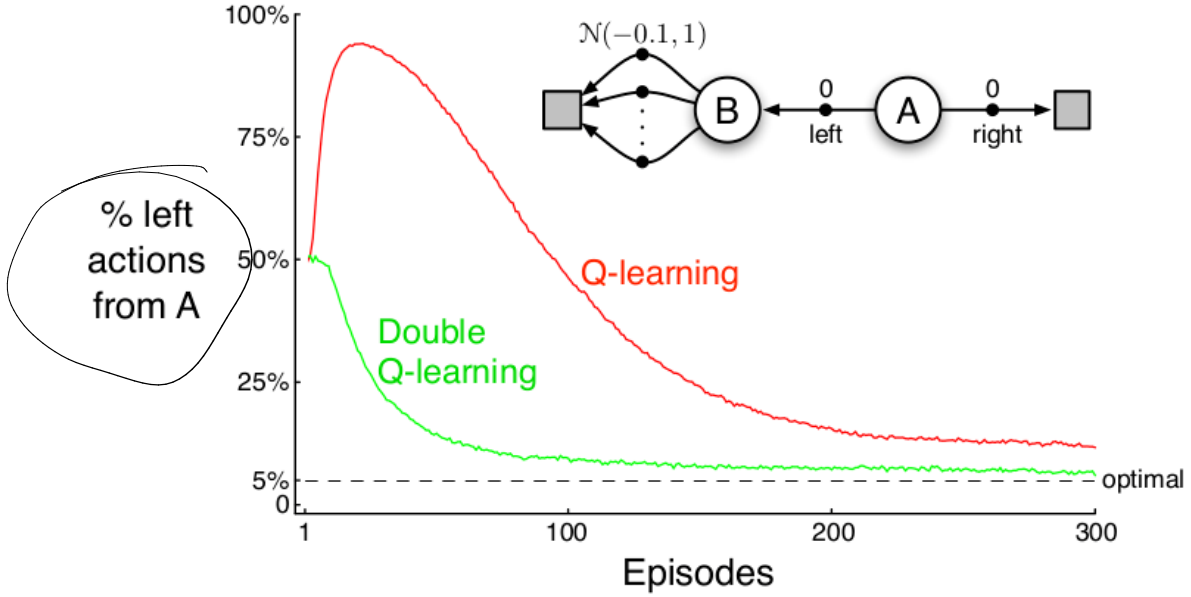
Big Problem: Maximization Bias

Even if all $Q(s', a')$ unbiased, $\max_{a'} Q(s', a')$ is biased!

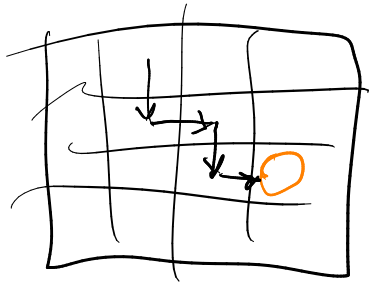
Solution: Double Q Learning $\quad Q_1, Q_2$

$$Q_1(s,a) \leftarrow Q_1(s,a) + \alpha \left(r + \gamma Q_2 \left(s', \underset{a'}{\operatorname{argmax}} Q_1(s', a') \right) - Q_1(s,a) \right)$$

switch off every step



Eligibility Traces



SARSA-λ

SARSA-λ

$Q(s,a), N(s,a) \leftarrow 0$

initialize s, a, r, s'

loop

$a' \leftarrow \text{argmax}\, Q(s',a) \text{ w.p. } 1-\epsilon, \quad \text{rand}(A) \text{ o.w.}$

$N(s,a) \leftarrow N(s,a) + 1$

$\delta \leftarrow r + \gamma Q(s',a') - Q(s,a)$

$Q(s,a) \leftarrow Q(s,a) + \alpha\delta N(s,a) \quad \forall s, a$

$N(s,a) \leftarrow \gamma\lambda N(s,a)$

$s \leftarrow s', \quad a \leftarrow a'$

$r \leftarrow \text{act!}(\text{env}, a)$

$s' \leftarrow \text{observe}(\text{env})$

N is a "decaying count of visits to (s,a) on this episode

Convergence

Convergence

- Q learning converges to optimal Q-values w.p. 1 (Sutton and Barto, p. 131)

Convergence

- Q learning converges to optimal Q-values w.p. 1
  (Sutton and Barto, p. 131)
- SARSA converges to optimal Q-values w.p. 1 ***provided that***
  $\pi \to \mathrm{greedy}$
  (Sutton and Barto, p. 129)

On vs Off-Policy

On vs Off-Policy

On Policy

On vs Off-Policy

On Policy

Off Policy

On vs Off-Policy

On Policy

SARSA:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\ (r + \gamma Q(s',a') - Q(s,a))$$

Off Policy

On vs Off-Policy

On Policy

SARSA:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\ (r + \gamma Q(s',a') - Q(s,a))$$

Off Policy

Q-learning:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\ (r + \gamma \max_{a'} Q(s',a') - Q(s,a))$$

On vs Off-Policy

On Policy

SARSA:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\ (r + \gamma Q(s',a') - Q(s,a))$$

Off Policy

Q-learning:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\ (r + \gamma \max_{a'} Q(s',a') - Q(s,a))$$

Will eligibility traces work with Q-learning?

On vs Off-Policy

On Policy

SARSA:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\ (r + \gamma Q(s',a') - Q(s,a))$$

Off Policy

Q-learning:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\ (r + \gamma \max_{a'} Q(s',a') - Q(s,a))$$

Will eligibility traces work with Q-learning?

Not easily

On vs Off-Policy

On Policy

SARSA:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\ (r + \gamma Q(s',a') - Q(s,a))$$

Off Policy

Q-learning:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\ (r + \gamma \max_{a'} Q(s',a') - Q(s,a))$$

Will eligibility traces work with Q-learning?

Not easily

Policy Gradient:

$$\theta \leftarrow \theta + \alpha \sum_{k=0}^{d} \nabla_\theta \log \pi_\theta(a_k \mid s_k) R(\tau)$$

Today

- Basic On- and Off-Policy **value based** model free RL algorithms
- Tricks for tabular value based RL algorithms
- Understanding of On- vs Off-Policy